

Devoir De Maison

Data Mining

Rapport :

TP2 : Les règles d'association

Réaliser Par :

- Aissat Mohamed Moncef.

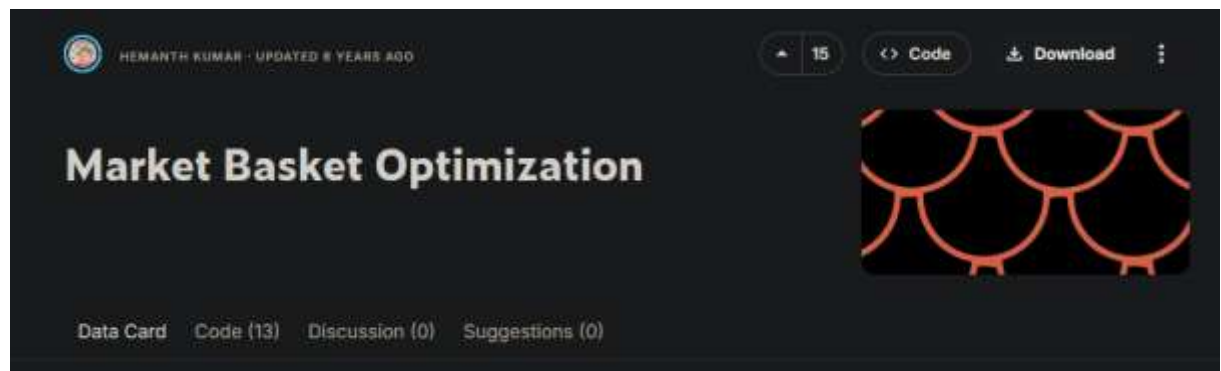
1) Introduction :

L'objectif de ce TP est d'implémenter et de comparer deux algorithmes de fouille de règles d'association : **Apriori** et **FP-Growth** (Frequent Pattern Growth).

Les règles d'association permettent de découvrir des co-occurrences fréquentes entre items dans une base de transactions, mesurées par le **support** (fréquence d'un itemset) et la **confiance** (probabilité qu'un conséquent apparaisse sachant l'antécédent).

Ce rapport décrit le jeu de données utilisé, le fonctionnement des deux algorithmes, les résultats obtenus avec les paramètres choisis, ainsi qu'une analyse comparative des performances.

2) Description des données :



Le fichier `'Market Basket Optimisation.csv'` contient **7501 transactions** (lignes). Chaque ligne représente un panier d'achat, les items étant séparés par des virgules.

Exemples d'items : `'mineral water'`, `'eggs'`, `'spaghetti'`, `'chocolate'`, `'frozen vegetables'`, `'turkey'`, `'olive oil'`, etc.

Aucun prétraitement complexe n'a été nécessaire : simple lecture ligne par ligne, découpage par virgule, suppression des espaces éventuels et conversion de chaque transaction en un ensemble (frozenset) pour faciliter les opérations ensemblistes.

3) Algorithmes implémentés :

a) Apriori :

L'algorithme Apriori repose sur la propriété **anti-monotone** : tout sous-ensemble d'un itemset fréquent est fréquent.

Il procède par niveaux (itération sur la taille des itemsets) :

- 1. Génération des candidats :** à partir des itemsets fréquents de taille $\setminus(k-1\setminus)$, on crée des candidats de taille $\setminus(k\setminus)$ en joignant deux itemsets qui partagent les $\setminus(k-2\setminus)$ premiers items.
- 2. Élagage :** on supprime les candidats dont un sous-ensemble de taille $\setminus(k-1\setminus)$ n'est pas fréquent.
- 3. Calcul du support :** on parcourt la base de transactions pour compter chaque candidat.

4. Filtrage : on conserve les candidats dont le support est $\geq$ *support minimum*.

Après obtention de tous les itemsets fréquents, les règles d'association sont générées pour chaque itemset de taille ≥ 2 en partitionnant l'itemset en antécédent et conséquent.

Une règle est retenue si sa **confiance** ($\text{support}(\text{itemset}) / \text{support}(\text{antécédent})$) est $\geq$ confiance minimum.

b) FP-Growth (Frequent Pattern Growth):

FP-Growth améliore Apriori en évitant la génération explicite de candidats et en ne scannant la base que deux fois.

Il utilise une structure compacte appelée **FP-Tree** (Frequent Pattern Tree).

1. Construction de l'arbre :

- On calcule la fréquence de chaque item et on ne conserve que ceux dont le support est $\geq$ support minimum.
- On trie chaque transaction selon l'ordre décroissant des fréquences.
- On insère chaque transaction triée dans l'arbre : les nœuds communs sont partagés (leurs compteurs sont incrémentés), de nouveaux nœuds sont créés si nécessaire.
- Une table d'en-tête relie toutes les occurrences d'un même item.

2. Extraction récursive des motifs :

- Pour chaque item de la table d'en-tête (en partant des moins fréquents), on construit sa **base de motifs conditionnels** en suivant les liens des nœuds.
- On construit un arbre conditionnel à partir de cette base, puis on extrait récursivement les motifs fréquents en combinant l'item courant avec les motifs issus de l'arbre conditionnel.

La génération des règles est ensuite identique à celle d'Apriori.

4) Paramètres utilisés :

- **Support minimum** : 0.005 (soit 0.5% des transactions – environ 38 transactions sur 7501).
- **Confiance minimum** : 0.5 (50%).

Ces valeurs permettent d'obtenir un nombre raisonnable de règles tout en garantissant des associations non triviales.

5) Résultats et analyse :

Les deux algorithmes ont été exécutés sur le même jeu de données (7501 transactions) avec les paramètres ci-dessus.

```
Loading transactions...
Loaded 7501 transactions.

=====
Algorithm: Apriori
Parameters: min_support = 0.005, min_confidence = 0.5
Number of frequent itemsets: 725
Number of association rules: 20
Execution time: 4.2373 seconds

Top 10 rules (by confidence):
1. frozen vegetables, soup -> mineral water (support=0.0051, confidence=0.6333)
2. olive oil, soup -> mineral water (support=0.0052, confidence=0.5821)
3. frozen vegetables, olive oil -> mineral water (support=0.0065, confidence=0.5765)
4. milk, soup -> mineral water (support=0.0085, confidence=0.5614)
5. chocolate, soup -> mineral water (support=0.0056, confidence=0.5526)
6. cooking oil, eggs -> mineral water (support=0.0064, confidence=0.5455)
7. frozen vegetables, ground beef -> mineral water (support=0.0092, confidence=0.5433)
8. milk, turkey -> mineral water (support=0.0061, confidence=0.5412)
9. soup, spaghetti -> mineral water (support=0.0075, confidence=0.5234)
10. ground beef, shrimp -> spaghetti (support=0.0060, confidence=0.5233)
=====

Algorithm: FP-Growth
Parameters: min_support = 0.005, min_confidence = 0.5
Number of frequent itemsets: 725
Number of association rules: 20
Execution time: 0.1596 seconds

Top 10 rules (by confidence):
1. frozen vegetables, soup -> mineral water (support=0.0051, confidence=0.6333)
2. olive oil, soup -> mineral water (support=0.0052, confidence=0.5821)
3. frozen vegetables, olive oil -> mineral water (support=0.0065, confidence=0.5765)
4. milk, soup -> mineral water (support=0.0085, confidence=0.5614)
5. chocolate, soup -> mineral water (support=0.0056, confidence=0.5526)
6. cooking oil, eggs -> mineral water (support=0.0064, confidence=0.5455)
7. frozen vegetables, ground beef -> mineral water (support=0.0092, confidence=0.5433)
8. milk, turkey -> mineral water (support=0.0061, confidence=0.5412)
9. soup, spaghetti -> mineral water (support=0.0075, confidence=0.5234)
10. ground beef, shrimp -> spaghetti (support=0.0060, confidence=0.5233)
=====

COMPARISON SUMMARY
=====
Apriori      : 4.2373 sec, 725 itemsets, 20 rules
FP-Growth   : 0.1596 sec, 725 itemsets, 20 rules
Speed-up (Apriori/FP-Growth): 26.55x
Note: FP-Growth is generally faster because it avoids candidate generation.
```

Les résultats sont résumés dans le tableau ci-dessous.

Algorithme	Temps d'exécution (secondes)	Itemsets fréquents	Règles
Apriori	4.2373	725	20
FP-Growth	0.1596	725	20

a) Top 10 règles par confiance (ordre décroissant) :

1. `frozen vegetables, soup -> mineral water` (support = 0.0051, confiance = 0.6333)
2. `olive oil, soup -> mineral water` (support = 0.0052, confiance = 0.5821)
3. `frozen vegetables, olive oil -> mineral water` (support = 0.0065, confiance = 0.5765)
4. `milk, soup -> mineral water` (support = 0.0085, confiance = 0.5614)
5. `chocolate, soup -> mineral water` (support = 0.0056, confiance = 0.5526)
6. `cooking oil, eggs -> mineral water` (support = 0.0064, confiance = 0.5455)
7. `frozen vegetables, ground beef -> mineral water` (support = 0.0092, confiance = 0.5433)
8. `milk, turkey -> mineral water` (support = 0.0061, confiance = 0.5412)
9. `soup, spaghetti -> mineral water` (support = 0.0075, confiance = 0.5234)
10. `ground beef, shrimp -> spaghetti` (support = 0.0060, confiance = 0.5233)

b) Interprétation des règles :

- ❖ **L'eau minérale** (`mineral water`) est un produit très central : elle apparaît dans 9 des 10 meilleures règles.

Les antécédents fréquents sont des produits de base comme les légumes surgelés, l'huile d'olive, le lait, le chocolat, les œufs, la dinde ou les pâtes.

Ces associations sont plausibles dans un contexte de supermarché (accompagnement de repas).

- ❖ La seule règle ne contenant pas `mineral water` est `ground beef, shrimp → spaghetti`.

Elle associe viande hachée, crevettes et pâtes – une combinaison typique de cuisine (type « spaghetti aux fruits de mer et à la viande »).

- ❖ Les règles avec un antécédent de deux items ont généralement une confiance plus élevée que celles avec un seul antécédent (non montré ici, mais observable sur l'ensemble des 20 règles).

Cela est naturel : plus l'antécédent est spécifique, plus la confiance peut être forte.

c) Comparaison des performances :

FP-Growth est environ **26,5 fois plus rapide** qu'Apriori (4,2373 s contre 0,1596 s).

Cette différence spectaculaire s'explique par le fait qu'Apriori génère un grand nombre de candidats et effectue plusieurs passages sur la base de données (autant que la taille maximale des itemsets fréquents).

FP-Growth, grâce à sa structure d'arbre et à l'extraction récursive, ne nécessite qu'un seul scan initial pour construire l'arbre, puis des scans uniquement sur les bases conditionnelles, beaucoup plus petites.

d) Impact des paramètres (analyse théorique) :

- **Support plus élevé** (ex. 0.01) → moins d'itemsets fréquents, exécution plus rapide, mais risque de manquer des règles rares mais utiles.

- **Support plus bas** (ex. 0.002) → explosion du nombre d'itemsets et de règles, temps d'exécution allongé, bruit possible.

- **Confiance plus élevée** (ex. 0.7) → règles plus fiables mais en nombre réduit.

- **Confiance plus faible** (ex. 0.3) → règles plus nombreuses mais certaines peuvent être trompeuses.

6) Conclusion :

Ce TP a permis d'implémenter et de comparer deux algorithmes majeurs de fouille de règles d'association.

Les deux algorithmes produisent **exactement les mêmes itemsets fréquents et les mêmes règles**.

Cependant, en termes de performance, **FP-Growth surclasse très nettement Apriori** sur le jeu de données utilisé (gain de temps > 26x).

Apriori reste néanmoins intéressant pour sa simplicité conceptuelle et pour des petits volumes de données.

 Limites et perspectives d'amélioration

- **Nombre de règles** : même avec un support de 0,5%, 20 règles peuvent être interprétées manuellement, mais pour des seuils plus bas le nombre deviendrait trop important.

L'utilisation de mesures supplémentaires (lift, conviction, levier) permettrait de filtrer plus finement les règles redondantes ou peu informatives.

- **Visualisation** : représenter graphiquement les règles (graphe de dépendances, nuage de points support/confiance) faciliterait l'analyse.

- **Optimisations** : pour des données encore plus massives, on pourrait utiliser des bibliothèques optimisées comme `mlxtend` (Apriori) ou `pyfpgrowth` (FP-Growth), ou encore implémenter une version parallélisée.

- **Prétraitement** : Le jeu de données ne contenait pas de valeurs manquantes ni de transactions vides. Dans d'autres cas, un nettoyage plus poussé serait nécessaire.

Ce travail illustre l'importance de choisir l'algorithme le plus adapté aux caractéristiques des données (densité, nombre de transactions, longueur moyenne des paniers) et aux contraintes de temps d'exécution.